

Implementation of AI Search Algorithms

Algorithms Implemented: Minimax, Alpha-Beta Pruning, Heuristic Alpha-Beta Search, and Monte-Carlo Tree Search (MCTS)

1. Introduction

Artificial Intelligence search algorithms are widely used in game playing, decision making, robotics, and optimization problems. In adversarial games such as Tic-Tac-Toe or Chess, an intelligent agent must choose the best move by predicting future game states.

In this project, four important AI search algorithms were implemented and tested:

1. Minimax Search Algorithm
2. Alpha-Beta Pruning
3. Heuristic Alpha-Beta Search
4. Monte-Carlo Tree Search (MCTS)

The algorithms were tested using the Tic-Tac-Toe game environment.

2. Minimax Algorithm

The Minimax algorithm is a recursive decision-making algorithm used in two-player games.

The main idea is:

- MAX player tries to maximize the score
- MIN player tries to minimize the score

The algorithm explores all possible future states of the game tree and chooses the move that gives the best possible outcome.

Advantages:

- Guarantees optimal move
- Easy to understand

Disadvantages:

- Very slow for large game trees
- High computational cost

Working of Minimax

- Generate all possible moves.

- Simulate each move recursively.
- Evaluate terminal states.
- Choose maximum score for MAX player.
- Choose minimum score for MIN player.

3. Alpha-Beta Pruning

Alpha-Beta Pruning is an optimization of the Minimax algorithm.

Instead of searching every branch of the tree, Alpha-Beta removes branches that cannot influence the final decision. This significantly improves performance.

Important Terms:

- Alpha = Best value found for MAX
- Beta = Best value found for MIN

Pruning occurs when:

$\text{Alpha} \geq \text{Beta}$

Advantages:

- Faster than Minimax
- Searches fewer nodes
- Produces same optimal result

4. Heuristic Alpha-Beta Search

In large games, searching the full game tree is impossible due to huge complexity.

Heuristic Alpha-Beta Search solves this problem by:

- Limiting the search depth
- Using an evaluation function (heuristic)

The heuristic function estimates how good a board position is.

Example:

- Reward positions close to winning
- Penalize opponent threats

Advantages:

- Faster execution
- Can handle larger games

- More practical in real-world AI systems

5. Monte-Carlo Tree Search (MCTS)

Monte-Carlo Tree Search is a probabilistic search algorithm that uses random simulations to estimate the best move.

MCTS became very popular after being used in modern game-playing systems such as AlphaGo.

The algorithm works in four phases:

1. Selection
2. Expansion
3. Simulation
4. Backpropagation

Selection:

The best child node is selected using the UCB formula.

Expansion:

A new child node is added to the tree.

Simulation:

A random game is played until the game ends.

Backpropagation:

Results are propagated back through the tree.

Advantages:

- Works well for very large search spaces
- Does not require full tree exploration
- Strong performance in complex games

6. Test Cases and Outputs

Several Tic-Tac-Toe board configurations were tested.

Sample Test Case:

Board:

X | O | X

0 | X |
| | O

Expected Best Move:
(2, 0)

Results:

- Minimax produced correct optimal move.
- Alpha-Beta produced same result with faster pruning.
- Heuristic Alpha-Beta selected strong moves efficiently.
- MCTS successfully identified high-quality moves using simulations.

7. Comparison of Algorithms

Algorithm	Optimal	Uses Pruning	Fast	Suitable for Large Games
Minimax	Yes	No	No	No
Alpha-Beta	Yes	Yes	Medium	Medium
Heuristic Alpha-Beta	Approximate	Yes	Yes	Yes
MCTS	Probabilistic	No	Very Fast	Excellent

8. Conclusion

This project successfully implemented four important Artificial Intelligence search algorithms.

The project demonstrated:

- Adversarial search
- Search optimization
- Heuristic evaluation
- Probabilistic decision making

Among all algorithms:

- Minimax gives guaranteed optimal results.
- Alpha-Beta improves efficiency.
- Heuristic Alpha-Beta is practical for larger games.
- MCTS is powerful for complex state spaces.

These algorithms form the foundation of many modern AI systems and game-playing

agents.

9. References

- Stuart Russell and Peter Norvig – Artificial Intelligence: A Modern Approach
- Introduction to Algorithms – Cormen
- Research papers on Monte-Carlo Tree Search
- Python Documentation